

Relatório de Correções e Segurança - Dinneer

- **Data:** 16/06/2026
- **Branch:** fix/editar-cep-jantar
- **Escopo:** correção de 2 bugs reportados + checkup de segurança do backend (Node/TypeScript em pdm_php/node-backend) e do app (Flutter em dinneer/), seguido de hardening de autenticação/autorização e limpeza de código.

1. Bugs corrigidos

Bug 1 e Bug 2 - Jantar duplicado + chat trocado (mesma causa raiz)

Sintomas relatados

- 1 Ao criar um jantar, ele aparecia **duplicado** na Home.
- 2 Um jantar novo com o **mesmo nome** de um anterior herdava o **chat do jantar antigo**.

Causa raiz

Nas queries `getCardapiosDisponiveis` e `getCardapiosPorCategoria` (`src/modules/cardapio/cardapio.service.ts`), o encontro era unido ao **local** em vez do **cardápio**:

```
-- ERRADO
INNER JOIN tb_encontro_dn d ON b.id_local = d.id_local
```

Como um mesmo local hospeda vários cardápios e vários encontros, o JOIN gerava **produto cartesiano** (NxM linhas -> duplicação) e pareava cada cardápio com um `id_encontro` **errado** (às vezes o do jantar antigo). Como o chat é indexado por `id_encontro` (`src/service/chat/chat_service.dart`), abrir o chat pela Home levava ao chat do encontro antigo.

Correção

```
-- CERTO
INNER JOIN tb_encontro_dn d ON a.id_cardapio = d.id_cardapio
```

As demais queries (`getMeusJantaresCriados`, `getMinhasReservas`) já uniam corretamente por `id_cardapio` e serviram de referência. A query de média de avaliações (`avaliacao.service.ts`) foi verificada e está correta (não há produto cartesiano nela).

Observação: mensagens já gravadas no Firebase sob o `id_encontro` errado permanecem lá; a correção evita novas trocas e faz a Home resolver o encontro certo, mas não migra histórico antigo.

2. Vulnerabilidades de segurança

2.1 Corrigidas

#	Severidade	Descrição	Correção	Arquivos
1	Crítica	Sem autenticação/autorização - todo endpoint era aberto e confiava no <code>id_usuario</code> enviado pelo cliente (IDOR generalizado: excluir conta alheia, agir como outro usuário etc.)	Token HMAC assinado emitido no login + middleware que popula <code>req.usuarioId</code> ; endpoints sensíveis derivam a identidade do token	<code>utils/token.ts</code> , <code>middlewares/auth.ts</code> , <code>app.ts</code> , <code>controllers</code> ; Flutter: <code>HttpService.dart</code> , <code>SessionService.dart</code> , <code>tela_login.dart</code>

#	Severidade	Descrição	Correção	Arquivos
2	Crítica	getUsuarios/getUsuario/createUsuario retornavam SELECT *, expondo senha e CPF de todos os usuários	Colunas explícitas, sem vl_senha/nu_cpf	modules/usuario/usuario.service.ts
3	Crítica	Senha guardada com criptografia reversível (AES256CBC, IV estático, chave default fraca)	Hash unidirecional bcrypt salgado + migração transparente das senhas legadas no 1º login	utils/password.ts, modules/usuario/usuario.service.ts
5	Média	CORS totalmente aberto (cors())	Allowlist por CORS_ORIGINS (libera apps mobile sem Origin)	app.ts, config/environment.ts
8	Baixa	Logs vazando dados - print(body) logava a senha no login; print("DEBUG HOME") e outros despejavam respostas completas	Removidos todos os print()/logs de payload	service/http/HttpService.dart, service/refeicao/cardapioService.dart, screens/tela_meus_jantares.dart, screens/perfil/components/tab_avaliacoes.dart, screens/tela_login.dart
-	Bônus	npm run build quebrado por 7 erros de tsc pré-existent (req/next não usados)	Parâmetros não usados prefixados com _; build agora limpo (tsc --noEmit -> exit 0)	app.ts + 6 controllers

2.2 Hardening de IDOR (identidade via token + checagem de dono)

Todos os endpoints que carregam identidade passaram a derivar o usuário do **token**, e ações restritas verificam **propriedade**:

Módulo	Operações	Regra aplicada
usuário	deleteUsuario, atualizarFotoPerfil	só a própria conta (id do token)
usuário	getUsuarios	exige token
encontro	reservar, cancelarReserva, verificarReserva, getMinhasReservas	identidade do token
encontro	aprovarReserva, rejeitarReserva, getParticipantes	só o anfitrião do encontro
cardápio	createJantar	dono = usuário do token
cardápio	updateJantar, deleteJantar	só o dono do cardápio
avaliação	createAvaliacao	autor = usuário do token
local	createLocal, getMeusLocais	identidade do token
local	deleteLocal	só o dono do local (antes: sem checagem - apagava local alheio e cascadeava encontros/cardápios)
local	getLocal, getLocais	exigem token (não usados pelo app)

Helpers de propriedade adicionados: getIdAnfitriaoPorEncontro (encontro), getIdDonoPorCardapio (cardápio), getIdDonoPorLocal (local).

> `getMeusJantaresCriados` e `getMediaUsuario` foram **mantidos públicos por parâmetro** porque alimentam o **perfil público** de outros usuários.

2.3 Pendentes (recomendações)

#	Severidade	Descrição	Recomendação
4	Crítica	Credenciais e chave de criptografia hardcoded como fallback em <code>config/environment.ts</code> (host, user/senha do banco, ENCRYPTION_KEY/IV) - estão no histórico do git	Rotacionar a senha do banco e remover os valores reais do código
6	Média	Geração de IDs manual e racy (<code>tb_sequence_dn</code> com <code>max+1</code>)	Migrar para SERIAL/sequence nativa do Postgres
7	Média	Chat sem checagem de participação no app; depende das <i>*security rules*</i> do Firebase	Verificar/endurecer as regras do Realtime Database
9	Info	Config Firebase web hardcoded em <code>main.dart</code> (chaves web não são segredo, mas a proteção real está nas rules + App Check)	Mover para config + garantir rules/App Check
10	Baixa	Sem <code>ratelimit</code> / headers de segurança (<code>helmet</code>); login sem <code>throttling</code>	Adicionar <code>helmet</code> e <code>ratelimit</code> no Express
-	Info	Módulos <code>imagem</code> e <code>notification</code> não revisados a fundo (o de notificação registra token FCM por usuário)	Revisar IDOR nesses módulos

3. Limpeza de código

- **Remoção de todos os comentários** dos arquivos `.dart` em `dinneer/lib/` (40 arquivos alterados), preservando código e anotações.
- Validado com `dart analyze lib`: **28 issues - idêntico ao baseline, zero erros** (nenhum código corrompido). Padrões sensíveis (URLs `http://`, interpolação com aspas aninhadas) verificados manualmente.

4. Ações necessárias antes do deploy

- 1 **Migração no Postgres** (obrigatória - o hash script tem ~168 chars):

```
ALTER TABLE tb_usuario_dn ALTER COLUMN vl_senha TYPE VARCHAR(255);
```

- 1 **Variáveis no .env** do backend:

```
AUTH_SECRET=<string longa e aleatória>
AUTH_EXPIRES_SECONDS=604800
CORS_ORIGINS=http://localhost:8080,https://seu-dominio
```

- 1 **Usuários precisam relogar** para obter o token (exigido nas operações protegidas). Senhas legadas migram para script automaticamente no primeiro login.
- 2 **Rotacionar a senha do banco** (item 4 das pendências).

5. Verificação realizada

- Backend: `npx tsc --noEmit -> exit 0` (compila limpo, inclusive a correção dos 7 erros pré-existent).
- App: `dart analyze lib -> 28 issues (todas info), zero erros`, igual ao baseline anterior às mudanças.